Task 4: Automation Design – Bank Operations Console

Tool: Playwright Test with TypeScript, using the Page Object Model as Playwright recommends.

Assumption: each setting sits in a stable wrapper with a test ID, for example `data-testid="daily-limit"`

1. Project structure

```
bank-ops-console-tests/
├── .github/workflows/playwright.yml   # CI: install, run, upload reports
├── playwright.config.ts               # projects, reporters, retries, trace
├── package.json
├── tsconfig.json
├── .env.example                       # BASE_URL etc. Real secrets live in CI, never in git
├── src/
│   ├── pages/
│   │   ├── ConsolePage.ts             # open a tab, find a setting, save, reload
│   │   └── SettingField.ts            # set and check one setting (text, checkbox, dropdown)
│   ├── fixtures/
│   │   └── index.ts                   # gives tests a consolePage, backs up and restores settings
│   └── api/
│       └── settingsApi.ts             # read and restore settings through the API
├── test-data/
│   └── settings.json                  # the 12 fields: tab, test ID, type, value
├── tests/
│   ├── auth.setup.ts                  # logs in once and saves the session
│   └── settings/
│       └── settings-persist.spec.ts
├── playwright/.auth/                  # saved login session (git-ignored)
├── playwright-report/                 # HTML report (git-ignored)
└── test-results/                      # traces, screenshots, videos, junit.xml (git-ignored)
```

Part	Job
`pages/`	Page objects. They act on the page and hold no test logic.
`fixtures/`	Set-up and clean-up that every test shares.
`api/`	Fast set-up and clean-up without going through the UI.
`test-data/`	Values. The test code holds none.
`tests/`	Tests that read like the scenario: update, save, reload, check.

Two page objects are enough. Every tab works the same way, so there's no class per tab and no method per field.

2. The flow (pseudocode)

```
FOR each row in settings.json                    # 12 rows across 4 tabs
    open row.tab, wait until it is selected
    find the setting by row.testId inside the open tab
    set it to row.value

click Save, wait for the save response and the success message
reload the page, wait for the settings to finish loading

FOR each row in settings.json
    open row.tab
    check the setting shows row.value            # soft check: collect every mismatch, then fail once
```

Example rows from `settings.json`:

Tab	Test ID	Type	New value
Transfer Limits	daily-limit	Text	20000

KYC	require-id-scan	Checkbox	On
Fees	fee-type	Dropdown	Flat

3. Locators strategy

The test finds the wrapper first, then the control inside it. `slot="field-145"` is never used.

Step	How	Why
1. Open tab	The visible tab panel, by role	A field on a hidden tab is never picked by mistake
2. Wrapper	Its `data-testid`	Agreed with developers; doesn't change when wording or language changes
3. Control	Text box, checkbox or dropdown inside the wrapper, by role	No dynamic ID involved

For example: `tabpanel → getByTestId('daily-limit') → getByRole('textbox')`.

`SettingField` handles each type the same way, whatever tab it's on:

Type	Set	Check after reload
Text	Fill in the value	Value equals the new value
Checkbox	Tick or untick	Checked state matches
Dropdown	Select the option	Selected option matches

- **Missing test ID:** the label is a temporary fallback until developers add one.
- **What I never use:** dynamic IDs, positions (`nth`) or long XPath.

4. Handling dynamic DOM

- **Locators are looked up again on every action:** a replaced element never goes "stale". Element references are never stored.
- **Tab switches:** after switching tab, the test waits until it's selected (`aria-selected="true"`).
- **Scrolling:** Playwright scrolls to each element and waits until it's ready before acting.
- **Assertions:** Playwright's checks retry until they pass or time out. There are no fixed waits.
- **Save:** the test waits for the save response and the success message.
- **Reload:** the test waits for the settings to finish loading before checking anything.

5. Test data strateg

- **Where test data values can live:** in `settings.json`. If they differ by environment, there's one file per environment.
- **Login:** a setup project logs in once and saves the session. Every test then starts already logged in.

What	Where it lives	Examples
Environment settings	`.env.qa` and `.env.uat`, picked with the `TEST_ENV` variable	`BASE_URL`, `API_URL`
Secrets	The CI secret store. Never in git, never printed in logs.	`CONSOLE_USER`, `CONSOLE_PASSWORD`
Test values	`test-data/settings.<env>.json`, picked with the same `TEST_ENV`	The 12 field values
Template	`.env.example`, committed with names only	Shows a new engineer what to set

How an environment is chosen:

- **Locally:** run with `TEST_ENV=uat npx playwright test`. The config loads `.env.uat`, sets the base URL
- **In CI:** the pipeline sets `TEST_ENV` and injects the secrets. No `.env` file is needed there.

6. After the test: clean-up and reports

Clean-up, handled by the fixture:

```
before the test:  back up the current settings (through the API)
run the test
after the test:   restore the backed-up settings (always, pass or fail)
```

Reports and evidence, set in `playwright.config.ts`:

Setting	Value	Why
Reporters	`list`, `html`, `junit`	Progress in the terminal, a readable report, and results for the CI dashboard
Trace	On first retry	A full step-by-step replay of a failed test
Screenshot and video	On failure only	Evidence for failures, without bloating passing runs
Retries	1 in CI, 0 locally	Absorbs infrastructure blips. A test that only passes on retry is flagged as flaky.
Projects	`setup` (login), then `chromium`	Logs in once and reuses the session

What happens after each run:

1. The fixture restores the original settings.
2. If the test failed, its trace, screenshot and video are saved in `test-results/`.
3. The HTML report shows each step and every mismatch, with the failure evidence attached.
4. CI uploads the report and test results as artifacts. A failure blocks the merge.

7. Scaling and CI

To add	What changes
A new tab	No code. Add its rows to the data file.
A new field	One row in the data file.
A new field type, such as a radio button	One entry in `SettingField`.
A tab that behaves differently	A small class for that tab only.

- **When it runs:** CI runs the suite on every merge, plus a nightly full run.
- **Tags:** `@smoke` and `@settings` make it possible to run a subset.
- **Shared settings:** these are shared, so this test doesn't run in parallel with other tests that change them.

8. Why Playwright

Reason	Benefit
Auto-waiting	Fewer flaky tests, no manual waits
UI and API in one tool	Set up data through the API and test the UI in the same test
All main browsers	Chromium, Firefox and WebKit (Safari) from one API

Fast and parallel	A fresh browser per test, run in parallel and split across CI machines
Everything built in	Test runner, fixtures, retries, reports and the trace viewer, with no extra libraries